

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ  
КОММУНИКАЦИЙ  
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение  
высшего образования

"Сибирский государственный университет телекоммуникаций и  
информатики"

Кафедра телекоммуникационных систем и вычислительных средств  
(ТС и ВС)

РАСЧЁТНО-ГРАФИЧЕСКАЯ РАБОТА  
по дисциплине  
*"Программирование"*  
СПИСОК АНАГРАММ

Студент:  
*Группа ИКС-532*

*Д.С. Окунев*

Предподаватель:  
*Преподаватель*

*А.И. Вейлер*

Новосибирск 2026 г.

# РАСЧЁТНО-ГРАФИЧЕСКАЯ РАБОТА

## 1.1 Задание

Разработать программу Anagram, выводящую список всех анаграмм для заданного слова. Программа принимает в качестве аргумента командной строки имя файла, содержащего слово на русском или английском языке. Анаграммами называются все возможные перестановки букв (независимо от того, являются ли они допустимыми словами). Количество вариантов равно факториалу количества букв.

Критерии оценки:

- «Удовлетворительно»: вывод неполного списка анаграмм; нет динамической памяти.
- «Хорошо»: полный список анаграмм; динамическая память; функция без рекурсии.
- «Отлично»: полный список анаграмм; рекурсивная функция; вывод на экран и в файл.

## 1.2 Анализ задачи

Анаграммы - слова, созданные путём перестановок букв из другого слова. Чтобы реализовать программу, генерирующую все анаграммы, напомним функцию, которая будет вызывать сама себя и генерировать слова следующим путём:

- Возьмем первый символ слова и поместим его в префикс (переменную, хранящую слово в процессе генерации)
- Оставшееся слово поместим в другую переменную и подадим на следующий рекурсивный вызов функции
- Проверим, остались ли во второй переменной еще символы, если да, то вызываем функцию еще раз, иначе записываем получившуюся анаграмму в массив с результатами
- Повторяем до тех пор, пока не пройдем циклом все буквы слова

Общая схема работы будет выглядеть следующим образом:



Рисунок 1 — Схема работы программы

### 1.3 Демонстрация работы

1. Введем слово Hello:

```

2026-05-28 08:56:45 [INFO] === Anagram Generator ===
2026-05-28 08:56:45 [INFO] Config file: config.conf
2026-05-28 08:56:45 [INFO] Input file: input.txt
=== Анаграммы ===
Слово: Hello

2026-05-28 08:56:45 [INFO] Generated 120 anagrams
Создано: 120 анаграмм
Время работы: 0 мс

Первые 100 анаграмм (из 120):
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
...

```

(а) Начало

```

lloHe
lleHo
lleoH
lloHe
lloeH
loHel
loHle
loeHl
loeHl
loeHl
lolHe
loleH
lolleH
oHell
oHell
oHlel
oHlle
... (и ещё 20)

Сохранено в: output.txt
2026-05-28 08:56:45 [INFO] Saved to output.txt
2026-05-28 08:56:45 [INFO] Finished

```

(б) Конец

Рисунок 2 — Проверка работы программы 1

2. Добавим к нему недопустимые символы (например число):

```

2026-05-28 09:15:07 [INFO] === Anagram Generator ===
2026-05-28 09:15:07 [INFO] Config file: config.conf
2026-05-28 09:15:07 [INFO] Input file: input.txt
=== Анаграммы ===
Слово: Hello7

2026-05-28 09:15:07 [ERROR] Word contains invalid characters

```

Рисунок 3 — Проверка работы программы 2

3. Подадим на вход слово на Кириллице:

```

2026-05-28 09:21:00 [INFO] === Anagram Generator ===
2026-05-28 09:21:00 [INFO] Config file: config.conf
2026-05-28 09:21:00 [INFO] Input file: input.txt
=== Анаграммы ===
Слово: Привет

2026-05-28 09:21:00 [INFO] Generated 720 anagrams
Создано: 720 анаграмм
Время работы: 0 мс

Первые 100 анаграмм (из 720):
Привет
Привте
Привет
Привет
Привте
Привте
Привте
Привте
Привте
Привте
...

```

(а) Начало

```

Петрви
Петирв
Петивр
Петври
Петвир
Птриве
Птриев
Птрвие
Птрвеи
... (и ещё 620)

Сохранено в: output.txt
2026-05-28 09:21:00 [INFO] Saved to output.txt
2026-05-28 09:21:00 [INFO] Finished

```

(б) Конец

Рисунок 4 — Проверка работы программы 3

4. Введем очень длинное слово:

```

2026-05-28 09:25:33 [INFO] === Anagram Generator ===
2026-05-28 09:25:33 [INFO] Config file: config.conf
2026-05-28 09:25:33 [INFO] Input file: input.txt
=== Анаграммы ===
Слово: Достопримечательность
2026-05-28 09:25:33 [ERROR] Too long word

```

Рисунок 5 — Проверка работы программы 4

## 5. Не подадим файл на вход:

```

2026-05-28 09:25:33 [INFO] === Anagram Generator ===
2026-05-28 09:25:33 [INFO] Config file: config.conf
2026-05-28 09:25:33 [INFO] Input file: input.txt
=== Анаграммы ===
Слово: Достопримечательность
2026-05-28 09:25:33 [ERROR] Too long word

```

Рисунок 6 — Проверка работы программы 5

## 6. Подадим неверный файл:

```

2026-05-28 09:29:26 [INFO] === Anagram Generator ===
2026-05-28 09:29:26 [INFO] Config file: config.conf
2026-05-28 09:29:26 [INFO] Input file: enter.txt
2026-05-28 09:29:26 [ERROR] Cannot open: enter.txt

```

Рисунок 7 — Проверка работы программы 6

Ссылка на репозиторий с работами курса: [GitHub](#).

## 1.4 Листинг

Примечание: так как отчет был сделать в latex, некоторые символы немного поплыли...

```

1 /* anagram_utils.h */
2 #ifndef ANAGRAM_UTILS_H
3 #define ANAGRAM_UTILS_H
4
5 #include <string>
6 #include <vector>
7
8 // Проверяем, состоит ли слово только из допустимых символов
9 bool is_valid_word(const std::wstring& word);
10
11 // Генератор анаграмм
12 bool generate_anagrams(const std::wstring& word, std::vector<
    std::wstring>& anagrams);

```

```
13
14 #endif
```

```
1  /* anagram_utils.cpp */
2  #include "anagram_utils.h"
3  #include "logger.h"
4  #include <algorithm>
5
6  static bool is_valid_char(wchar_t ch) { // Проверка символа
    на корректность
7      if ((ch >= L'a' && ch <= L'z') || (ch >= L'A' && ch <= L'Z'
8          ')) {
9          return true;
10     }
11     if ((ch >= L'a' && ch <= L'я') || (ch >= L'A' && ch <= L'
12     Я') ||
13         ch == L'ё' || ch == L'Ё') {
14         return true;
15     }
16     return false;
17 }
18
19 bool is_valid_word(const std::wstring& word) { // Проверка
    слова
20     if (word.empty()) {
21         return false;
22     }
23
24     for (size_t i = 0; i < word.length(); ++i) {
25         if (!is_valid_char(word[i])) {
26             return false;
27         }
28     }
29     return true;
30 }
31
32 static void generate_recursive(const std::wstring& prefix,
33                               const std::wstring& remaining,
```

```

32         std::vector<std::wstring>&
result) { // === Генератор анаграмм ===
33     if (remaining.empty()) {
34         result.push_back(prefix);
35         return;
36     }
37
38     for (size_t i = 0; i < remaining.length(); ++i) {
39         generate_recursive(
40             prefix + remaining[i],
41             remaining.substr(0, i) + remaining.substr(i + 1),
42             result
43         );
44     }
45 }
46
47 bool generate_anagrams(const std::wstring& word, std::vector<
std::wstring>& anagrams) { // Формируем слово и рекурсивно
генерируем анаграммы
48     anagrams.clear();
49
50     if (word.empty()) {
51         log_error("Empty word");
52         return false;
53     }
54
55     generate_recursive(L"", word, anagrams);
56
57     log_info("Generated " + std::to_string(anagrams.size()) +
" anagrams");
58     return true;
59 }

```

```

1 /* config.h */
2 #ifndef CONFIG_H
3 #define CONFIG_H
4
5 #include <string>
6

```

```

7 // Загружаем конфиг из файла
8 bool config_load(const std::string& filename);
9
10 // Получаем значение по ключу с ( дефолтом)
11 std::string config_get(const std::string& key, const std::
    string& default_value = "");
12
13 // Проверяем наличие ключа
14 bool config_has(const std::string& key);
15
16 // Устанавливаем значение
17 void config_set(const std::string& key, const std::string&
    value);
18
19 // Очищаем все настройки
20 void config_clear();
21
22 #endif

```

```

1 /* config.cpp */
2 #include "config.h"
3 #include <fstream>
4 #include <algorithm>
5 #include <unordered_map>
6
7 static std::unordered_map<std::string, std::string> g_config;
8
9 static std::string trim(const std::string& s) { // Удаляем
    невидимые символы табы(, пробелы, переводы строки)
10     size_t start = s.find_first_not_of(" \t\r\n");
11     if (start == std::string::npos) return "";
12     size_t end = s.find_last_not_of(" \t\r\n");
13     return s.substr(start, end - start + 1);
14 }
15
16 bool config_load(const std::string& filename) { // Загружаем
    конфиг из файла
17     std::ifstream file(filename);
18     if (!file.is_open()) return false;

```

```

19
20     std::string line;
21     while (std::getline(file, line)) {
22         line = trim(line);
23         if (line.empty() || line[0] == '#') continue;
24
25         size_t pos = line.find('=');
26         if (pos == std::string::npos) continue;
27
28         std::string key = trim(line.substr(0, pos));
29         std::string value = trim(line.substr(pos + 1));
30
31         if (!key.empty()) {
32             g_config[key] = value;
33         }
34     }
35     return true;
36 }
37
38 std::string config_get(const std::string& key, const std::
    string& default_value) { // Получаем значение по ключу с (
    дефолтом)
39     auto it = g_config.find(key);
40     return (it != g_config.end()) ? it->second : default_value
    ;
41 }
42
43 bool config_has(const std::string& key) { // Проверяем
    наличие ключа
44     return g_config.find(key) != g_config.end();
45 }
46
47 void config_set(const std::string& key, const std::string&
    value) { // Устанавливаем значение
48     g_config[key] = value;
49 }
50
51 void config_clear() { // Очищаем все настройки
52     g_config.clear();
53 }

```

```

1  /* logger.h */
2  #ifndef LOGGER_H
3  #define LOGGER_H
4
5  #include <iostream>
6  #include <string>
7  #include <fstream>
8  #include <chrono>
9  #include <ctime>
10 #include <iomanip>
11 #include <sstream>
12
13 enum LogLevel { LOG_INFO, LOG_WARNING, LOG_ERROR };
14
15 inline static std::ofstream g_log_file;
16 inline static LogLevel g_min_level = LOG_INFO;
17 inline static bool g_console_output = true;
18
19 inline std::string timestamp() { // Получение времени и его
    форматирование
20     auto now = std::chrono::system_clock::now();
21     auto t = std::chrono::system_clock::to_time_t(now);
22     std::ostringstream oss;
23     oss << std::put_time(std::localtime(&t), "%Y-%m-%d %H:%M:%
    S");
24     return oss.str();
25 }
26
27 inline const char* level_str(LogLevel level) { // Ур.
    логирования в строку
28     switch (level) {
29         case LOG_INFO:      return "INFO";
30         case LOG_WARNING:   return "WARNING";
31         case LOG_ERROR:     return "ERROR";
32         default:            return "UNKNOWN";
33     }
34 }
35
36 inline void log(LogLevel level, const std::string& message) {
    // Формирование сообщений
37     if (level < g_min_level) return;

```

```

38
39     std::string line = timestamp() + " [" + level_str(level) +
    "]" + message;
40
41     if (g_log_file.is_open()) {
42         g_log_file << line << std::endl;
43         g_log_file.flush();
44     }
45
46     if (g_console_output) {
47         if (level == LOG_ERROR) {
48             std::cerr << line << std::endl;
49         } else {
50             std::cout << line << std::endl;
51         }
52     }
53 }
54
55 inline void log_info(const std::string& msg) { log(LOG_INFO,
    msg); }
56 inline void log_warning(const std::string& msg) { log(
    LOG_WARNING, msg); }
57 inline void log_error(const std::string& msg) { log(LOG_ERROR,
    msg); }
58
59 inline void logger_init(const std::string& filename) { //
    Открытие файла с логами
60     if (!filename.empty()) {
61         g_log_file.open(filename, std::ios::out | std::ios::
    app);
62     }
63 }
64
65 inline void logger_set_level(LogLevel level) { //
    Устанавливаем минимальный уровень
66     g_min_level = level;
67 }
68
69 inline void logger_set_console(bool enabled) { // Выводить в
    консоль или нет
70     g_console_output = enabled;
71 }

```

```

72
73 inline void logger_close() { // Закрываем файл с логами
74     if (g_log_file.is_open()) {
75         g_log_file.close();
76     }
77 }
78
79 #endif

```

```

1  /* main.cpp */
2  #include "config.h"
3  #include "logger.h"
4  #include "anagram_utils.h"
5
6  #include <iostream>
7  #include <fstream>
8  #include <vector>
9  #include <chrono>
10 #include <locale>
11 #include <codecvt>
12 #include <algorithm>
13
14 std::string wstring_to_utf8(const std::wstring& wstr) {
15     static std::wstring_convert<std::codecvt_utf8<wchar_t>>
16     conv;
17     try {
18         return conv.to_bytes(wstr);
19     } catch (...) {
20         return "[Error]";
21     }
22 }
23
24 LogLevel parse_log_level(const std::string& level) {
25     std::string upper = level;
26     std::transform(upper.begin(), upper.end(), upper.begin(),
27     ::toupper);
28     if (upper == "ERROR") return LOG_ERROR;
29     if (upper == "WARNING") return LOG_WARNING;
30     return LOG_INFO;

```

```

29 }
30
31 int main(int argc, char* argv[]) {
32     std::locale::global(std::locale(""));
33
34     // Загрузка конфига если( указан)
35     std::string config_file = "config.conf";
36     for (int i = 1; i < argc; i++) {
37         if (std::string(argv[i]) == "--config" && i + 1 < argc
38     ) {
39             config_file = argv[++i];
40             break;
41         }
42
43     config_load(config_file);
44
45     // Аргументы командной строки имеют приоритет
46     for (int i = 1; i < argc; i++) {
47         std::string arg = argv[i];
48         if (arg.find('=') != std::string::npos) {
49             size_t pos = arg.find('=');
50             config_set(arg.substr(0, pos), arg.substr(pos + 1)
51 );
52         }
53
54         // Инициализация логгера из конфига
55         std::string log_file = config_get("LOG_FILE", "app.log");
56         std::string log_level = config_get("LOG_LEVEL", "INFO");
57         bool console_log = config_get("CONSOLE_LOG", "true") == "
58 true";
59
60         logger_init(log_file);
61         logger_set_level(parse_log_level(log_level));
62         logger_set_console(console_log);
63
64         // Проверка входного файла
65         std::string input_file;
66         for (int i = 1; i < argc; i++) {
67             if (argv[i][0] != '-' && std::string(argv[i]).find('=')
68 ) == std::string::npos) {

```

```

67         input_file = argv[i];
68         break;
69     }
70 }
71
72 if (input_file.empty()) {
73     log_error("No input file specified");
74     std::cout << "Usage: " << argv[0] << " <input_file>
75     [--config file.conf] [KEY=VALUE]\n";
76     logger_close();
77     return 1;
78 }
79
80 log_info("=== Anagram Generator ===");
81 log_info("Config file: " + config_file);
82 log_info("Input file: " + input_file);
83
84 // Чтение слова
85 std::wifstream f(input_file);
86 if (!f.is_open()) {
87     log_error("Cannot open: " + input_file);
88     logger_close();
89     return 1;
90 }
91
92 std::wstring word;
93 f >> word;
94 f.close();
95
96 if (word.empty()) {
97     log_error("Empty file");
98     logger_close();
99     return 1;
100 }
101
102 std::cout << "=== Анаграммы === " << std::endl;
103 std::cout << "Слово: " << wstring_to_utf8(word) << "\n\n";
104
105 if (word.length() > 10) {
106     log_error("Too long word");
107     logger_close();
108     return 1;

```

```

108     }
109
110     if (!is_valid_word(word)) {
111         log_error("Word contains invalid characters");
112         logger_close();
113         return 1;
114     }
115
116     // Генерация анаграмм
117     auto start = std::chrono::steady_clock::now();
118     std::vector<std::wstring> anagrams;
119
120     if (!generate_anagrams(word, anagrams)) {
121         log_error("Generation failed");
122         logger_close();
123         return 1;
124     }
125
126     auto duration = std::chrono::duration_cast<std::chrono::
milliseconds>(
127         std::chrono::steady_clock::now() - start);
128
129     // Результаты
130     size_t total = anagrams.size();
131     int max_display = std::stoi(config_get("MAX_DISPLAY", "100
"));
132     size_t display = std::min(total, (size_t)max_display);
133
134     std::cout << "Создано: " << total << " анаграмм\n";
135     std::cout << "Время работы: " << duration.count() << " мс\
n";
136
137     if (total > 0) {
138         std::cout << "\n";
139         if (total <= (size_t)max_display) {
140             std::cout << "Все анаграммы:\n";
141         } else {
142             std::cout << "Первые " << display << " анаграмм
из( " << total << "):\n";
143         }
144
145         for (size_t i = 0; i < display; i++) {

```

```

146         std::cout << " " << wstring_to_utf8(anagrams[i])
    << "\n";
147     }
148
149     if (total > display) {
150         std::cout << " ... и( ещё " << (total - display)
    << ")\n";
151     }
152 }
153
154 /* Сохранение в файл */
155 std::string output_file = config_get("OUTPUT_FILE", "
    output.txt");
156 std::wofstream out(output_file);
157
158 if (out.is_open()) {
159     for (const auto& a : anagrams) {
160         out << a << L"\n";
161     }
162     out.close();
163     std::cout << "\Сохранен в: " << output_file << "\n";
164     log_info("Saved to " + output_file);
165 } else {
166     log_warning("Cannot save to " + output_file);
167 }
168
169 log_info("Finished");
170 logger_close();
171 return 0;
172 }

```

```

1 /* test_anagram.cpp */
2 #include <stdarg.h>
3 #include <stddef.h>
4 #include <setjmp.h>
5 extern "C" {
6 #include <cmocka.h>
7 }
8 #include <string>

```

```

9 #include <vector>
10 #include "anagram_utils.h"
11 #include "config.h"
12
13 // Слово на английском
14 static void test_valid_word_latin(void**) {
15     assert_true(is_valid_word(L"hello"));
16     assert_true(is_valid_word(L"WORLD"));
17     assert_true(is_valid_word(L"AbCdEf"));
18 }
19
20 // Слово на русском
21 static void test_valid_word_cyrillic(void**) {
22     assert_true(is_valid_word(L"привет"));
23     assert_true(is_valid_word(L"МОСКВА"));
24     assert_true(is_valid_word(L"ёлка"));
25     assert_true(is_valid_word(L"Ёжик"));
26 }
27
28 // Пустое слово
29 static void test_invalid_word_empty(void**) {
30     assert_false(is_valid_word(L""));
31 }
32
33 // Слово с недопустимыми символами
34 static void test_invalid_word_with_digits(void**) {
35     assert_false(is_valid_word(L"abc1"));
36     assert_false(is_valid_word(L"123"));
37 }
38
39 // Слово с пробелом
40 static void test_invalid_word_with_spaces(void**) {
41     assert_false(is_valid_word(L"hello world"));
42     assert_false(is_valid_word(L" "));
43 }
44
45 // Слово со знаком
46 static void test_invalid_word_with_punctuation(void**) {
47     assert_false(is_valid_word(L"hello!"));
48     assert_false(is_valid_word(L"test-case"));
49     assert_false(is_valid_word(L"what?"));
50 }

```

```

51
52 // Слово из одной буквы генерация()
53 static void test_anagrams_single_char(void**) {
54     std::vector<std::wstring> result;
55     assert_true(generate_anagrams(L"a", result));
56     assert_int_equal((int)result.size(), 1);
57     assert_true(result[0] == L"a");
58 }
59
60 // Слово с двумя буквами генерация()
61 static void test_anagrams_two_chars(void**) {
62     std::vector<std::wstring> result;
63     assert_true(generate_anagrams(L"ab", result));
64     assert_int_equal((int)result.size(), 2);
65 }
66
67 // Слово с тремя буквами генерация()
68 static void test_anagrams_three_chars_count(void**) {
69     std::vector<std::wstring> result;
70     assert_true(generate_anagrams(L"abc", result));
71     assert_int_equal((int)result.size(), 6);
72 }
73
74 // Слово с четырьмя буквами генерация()
75 static void test_anagrams_four_chars_count(void**) {
76     std::vector<std::wstring> result;
77     assert_true(generate_anagrams(L"abcd", result));
78     assert_int_equal((int)result.size(), 24);
79 }
80
81 // Пустое слово генерация()
82 static void test_anagrams_empty_word(void**) {
83     std::vector<std::wstring> result;
84     assert_false(generate_anagrams(L"", result));
85 }
86
87 int main(void) {
88     const struct CMUnitTest tests[] = {
89         cmocka_unit_test(test_valid_word_latin),
90         cmocka_unit_test(test_valid_word_cyrillic),
91         cmocka_unit_test(test_invalid_word_empty),
92         cmocka_unit_test(test_invalid_word_with_digits),

```

```
93     cmocka_unit_test(test_invalid_word_with_spaces),
94     cmocka_unit_test(test_invalid_word_with_punctuation),
95
96     cmocka_unit_test(test_anagrams_single_char),
97     cmocka_unit_test(test_anagrams_two_chars),
98     cmocka_unit_test(test_anagrams_three_chars_count),
99     cmocka_unit_test(test_anagrams_four_chars_count),
100    cmocka_unit_test(test_anagrams_empty_word),
101    };
102    return cmocka_run_group_tests(tests, NULL, NULL);
103 }
```